Indian Institute of Technology, Jodhpur

Assignment 2

for the subject

Natural Language Understanding

Submitted by

M25MAC016

Zobiya Khalid

MTech DCS

Report for problem 1

Learning Word Embeddings from IIT Jodhpur Data

1 Dataset Collection

Data was collected from the IIT Jodhpur website. Text was directly copied and pasted into a txt file directly from the website.

File names and sources are listed below

1. Iitj_departments_homepages.txt: All Departments home pages, entire research sections, news sections, highlights, motivation and vision pages
2. Iitj_homepage_campuslife_research.txt: IITJ website homepage, research highlights, news section, chairman and director, Campus life section
3. Iitj_schools: All Schools homepages, research section, academics and FAQ sections
4. Iitj_department_of_cs: Homepage and all sections and subsections of from the webpages of department of Computer Science, including research areas and labs, academics, programs offered, courses, people.
5. Iitj_academic_regulations: includes all the text on the iitj website academic regulations section.

Available pdfs of academic regulation documents from the iitj website academic regulations section, were downloaded. These pdfs included academic regulations since 2019, notifications and includes the current regulation document as well. In total these are 10 documents.

Academic regulations pdfs were also separately downloaded from the link provided on department of computer science's homepage for all four, BTech CS, BTech AI, Mtech CS, and Mtech AI. The files are named as regulations_btech_ai, regulations_btech_cs, regulations_mtech_ai, regulations_mtech_cs.

Other than these, pdfs of course list offered by department of CS and course list specialization for Btech students were downloaded from the course section of department of CS. The file names are: course_list_offeredby_department_CS, course_list_specialization_btech_cs.

Course syllabus was also downloaded. These files include course_syllabus_btechCS, course_syllabus_btechAI, course_syllabus_mtechCS_phdCS, course_syllabus_mtechAI, course syllabus_mtechDCS.

2 Dataset Processing

The pdfs were converted into a .txt format. All txt files were saved in a folder at the same location. These files were converted into lowercase. All punctuations are numbers were removed, and each word was treated as one token.

These text files were then combined to form one corpus file.

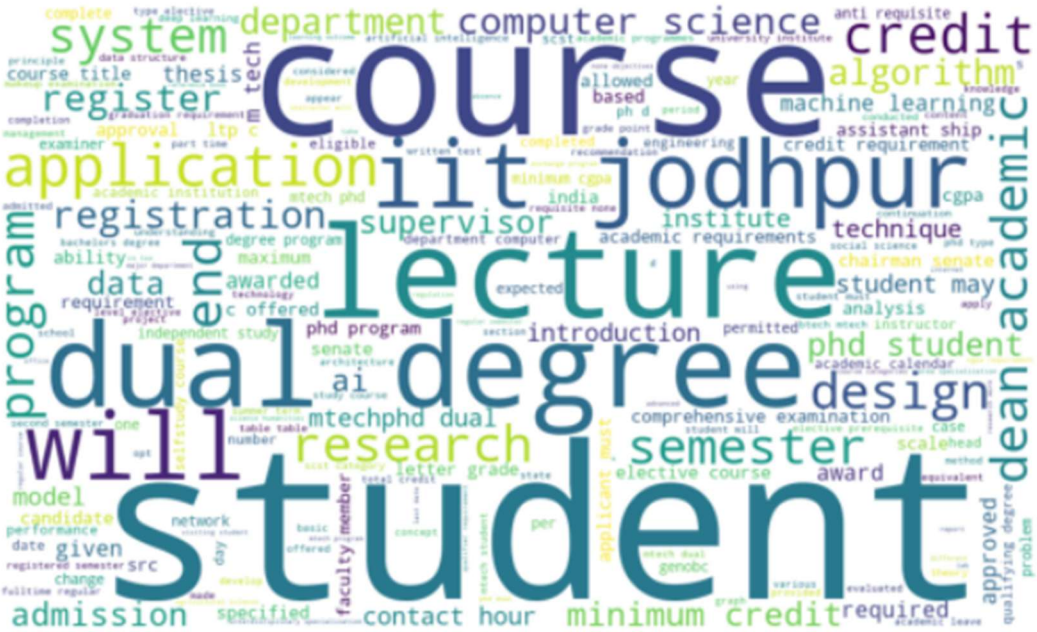
Due to conversion from pdf to text file some words remain combined like ‘aredesignedtodevelopthe', 'toproducequalityprofessionals'. To address this, wordninja library was used to split the words with length greater than 12 using English dictionary.
This resulted in a decline from 6735 words that occur only once to 4444.

3 Dataset Statistics

Total number of tokens: 191186
Total number of unique tokens (vocabulary size): 11899
Total number of documents: 48

Most frequent words include:
‘the’ which occurs 11625 times, 'of' 7743 times, 'and', 5351 times, 'a', 4319 times as shown in the histogram in figure 2.

Rare words which occur only include 'dignity', 'unfair', 'confidentiality' and more


course
student
lecture
dual degree
will
iit jodhpur
department
computer science
credit
system
algorithm
machine learning
register
application
registration
supervisor
institute
technique
academic
program
data
research
semester
design
phd student
dean
model
admission
contact hour
minimum
credit
required
approved
course title
thesis
tech
end
awarded
student may
introduction
phd program
mtechphd dual
comprehensive examination
elective course
award
given
faculty
member
ai
senate
allowed
based
credit requirement
assistant ship
academic requirements
chairman
senate
analysis
candidate
performance
network
src
specified
letter grade
scalehead
case
approval
ability
maximum
degree program
department computer
independent study
requirement
c offered
expected
permitted
instructor
academic calendar
genobc
applicant must
qualifying degree
problem
day
change
date
registered semester
fulltime regular
made
offered
table table
number
total credit
student will
elective prerequisite
method
equivalent
academic leave
evaluated
various
provided
grade
concept
basic
attach program
attach student
attach dual
teach student
teach program
student must
apply
instructor
section
technology
level elective
project
school
bachelor's degree
understanding
social science
phd type
course
using
recommendation
recommendation
written test
completed
eligible
engineering
grade point
conducted contents
knowledge
graduation requirement
completion
management
examiner
part time
person
ph d
year
appear
considered
development
data structure
principle
complete
type elective
anti requisite
artificial intelligence
academic programmes
university institute
second semester
one
table table
number
student will
elective prerequisite
case

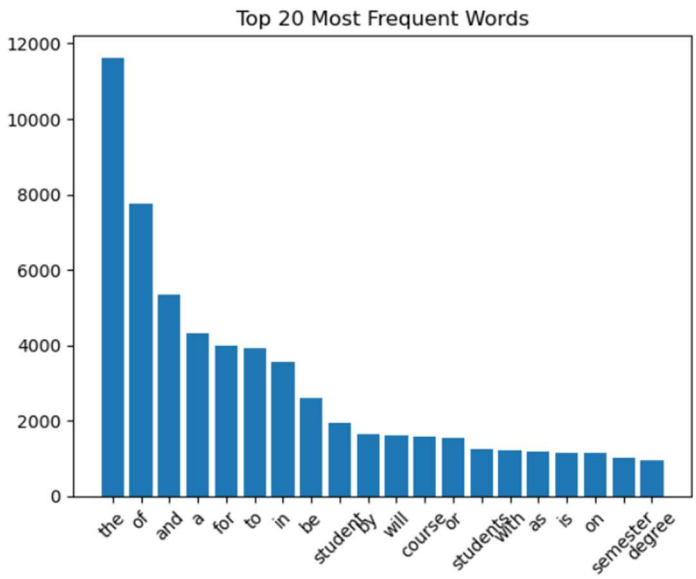


Figure 2. Word token histogram

4 Model Training

Two models were trained

Continuous Bag of Words (CBOW) with negative sampling and Skip-gram with Negative Sampling. For each model, experiment with:
(i) Embedding dimension
(ii) Context window size
(iii) Number of negative samples

4.1 Experimentations

4.1.1 CBOW Experiments:
A total of 27 experiments were done on CBOW with the combination of the following metrics for 2 epochs each

Embedding_dims = [50, 100, 150]
Window_sizes = [2, 4, 6]
Negative_samples_list = [2, 5, 6]

The observations are as follows:

1. Effect of negative samples: As negative samples increases loss also increases. Increasing the number of negative samples leads to higher loss values due to additional penalty terms, but improves the model's ability to distinguish correct and incorrect word contexts
2. Effect of window size: Loss decreases slightly with a larger window

3. Effect of embedding dimension: Increasing embedding dimension did not significantly affect performance in early training epochs. Higher dimensions may require more training to show benefits.
4. Lowest loss was observed in embedding dimension 150, window size 6, negative samples 2, closely followed by embedding dimension 100, window size 6, negative samples 2

```
Running: dim=150, window=6, neg=2
Epoch 1, Loss: 368644.8544
Epoch 2, Loss: 264227.1985
Running: dim=100, window=6, neg=2
Epoch 1, Loss: 368407.9440
Epoch 2, Loss: 262953.3425
```

4.1.2 Skip-gram Experiments

A total of 27 experiments were done with the combination of the following metrics for 2 epochs each

Embedding_dims = [50, 100, 150]

Window_sizes = [2, 4, 6]

Negative_samples_list = [2, 5, 8]

The observations are as follows:

1. Effect of negative samples: As the number of negative samples increases, the loss also increases.
2. Effect of window size: Loss increases with a larger window
3. Effect of embedding dimension: Increasing embedding dimension did not significantly affect performance in early training epochs.
4. Lowest loss was observed in embedding dimension 150, window size 2, negative samples 2, closely followed by embedding dimension 100, window size 2, negative samples 2.

```
Running Skip-gram: dim=150, window=2, neg=2
Epoch 1, Loss: 1199007.7093
Epoch 2, Loss: 792587.4122
Running Skip-gram: dim=100, window=2, neg=2
Epoch 1, Loss: 1201187.4990
Epoch 2, Loss: 794214.3103
```

4.2 Final Model Training

Based on the above experimentation observations final CBOW and Skip-gram models were trained with the following hyperparameters:

CBOW: The final model was trained with an embedding dimension of 100, window size of 6, and 1 negative sample. Training was carried out for 6 epochs. The final training loss at epoch 6 was 122,011.7120**.**

```
Running: dim=100, window=6, neg=1
Epoch 1, Loss: 239071.8705
Epoch 2, Loss: 180791.5395
Epoch 3, Loss: 152554.8486
Epoch 4, Loss: 135734.7071
Epoch 5, Loss: 127072.6340
Epoch 6, Loss: 122011.7120
```

Skip-gram: The model was trained with an embedding dimension of 100, window size of 4, and 2 negative samples. Due to higher computational cost, the model was trained for 3 epochs. The final training loss at epoch 3 was 1,678,918.4038.

```
Epoch 1, Loss: 2983372.4558
Epoch 2, Loss: 1837335.2343
Epoch 3, Loss: 1678918.4038
```

Loss reduced significantly for both the models after each epoch

5 Semantic Analysis

5.1 Cosine Similarity

Top 5 nearest neighbours for the words research, student, phd, exam were generated for CBOW and Skipgram the results are as follows:

```
CBOW Nearest neighbours for 'research':
faculty : 0.9394
areas : 0.9311
challenges : 0.9280
knowledge : 0.9256
ability : 0.9255

CBOW Nearest neighbours for 'student':
who : 0.8755
can : 0.8605
may : 0.8415
an : 0.8402
is : 0.8354

CBOW Nearest neighbours for 'phd':
eligible : 0.9462
semesters : 0.9454
allowed : 0.9436
fellowship : 0.9434
status : 0.9431

CBOW Nearest neighbours for 'exam':
step : 0.9971
applying : 0.9969
scheduled : 0.9967
yet : 0.9966
supplementary : 0.9966
```

```
Skip-gram Nearest neighbours for 'research':
supervisors : 0.9313
thesis : 0.9163
proposal : 0.9162
ability : 0.9154
through : 0.9151

Skip-gram Nearest neighbours for 'student':
who : 0.8812
can : 0.8530
leave : 0.8339
may : 0.8302
an : 0.8279

Skip-gram Nearest neighbours for 'phd':
programs : 0.9448
admissions : 0.9436
admission : 0.9403
candidates : 0.9399
fees : 0.9391

Skip-gram Nearest neighbours for 'exam':
topic : 0.9973
tests : 0.9971
middle : 0.9970
satisfy : 0.9969
applied : 0.9968
```

For the word:

- “research”: both models returned semantically relevant terms such as study, thesis, proposal, knowledge, and analysis. Skip-gram produced slightly more context-specific academic terms like thesis, proposal, etc indicating better capture of fine-grained relationships.
- “student”: The nearest neighbours included words like who, can, may, an, and is. These may not seem semantically meaningful, but this is expected given the nature of the dataset. Since the corpus was taken from the IIT Jodhpur website, which mainly contains instructions and informational content for students, the word “student” frequently appears in phrases such as “students can…”, “students may…”, etc, the association makes sense.
- “phd”: Both models produced highly relevant academic terms such as admissions, programs, candidates, fellowship, and eligibility. This indicates that the models effectively captured domain-specific relationships within the corpus.
- “exam”: The nearest neighbours included words like tests, schedule, applying, topic, and supplementary. These are strongly related to exam processes, suggesting that both models learned meaningful contextual associations.

Overall, CBOW tends to produce more general associations (e.g., knowledge, ability). While skip-gram captures more specific and context-driven relationships (e.g., thesis, proposal, admissions).

5.2 Analogy Experiments

Analogy tasks were performed to evaluate how well the learned embeddings capture semantic relationships of the form *a : b :: c : ?*. The results for both CBOW and Skip-gram models are summarized below.

The observations are as follows:

1. Academic hierarchy and program relationships:
   For analogies like *"UG : BTech :: PG : ?"*, both models produced relevant outputs such as *MSc* and *MTech*, indicating that the embeddings successfully captured relationships between different academic levels and programs.

2. Role-based relationships (Student Professor):
   In the analogy *"Student : Study :: Professor : ?"*, both models returned partially relevant words such as *lectures* and *design*, but also unrelated terms like *fractal*. This suggests that while some contextual association is captured, the relationship is not learned very cleanly, likely due to limited or noisy context in the dataset.

3. Mode of study (Full-time vs Part-time):
   For *"fulltime : regular :: parttime : ?"*, the outputs included terms like *selfstudy* and *independent*, which are reasonably meaningful. This indicates that the model has learned some distinction between structured and flexible modes of study.

4. Mode of study (Credit and Audit):
   Both should Ideally be credit : course :: audit : course, but both models generated *independent*, *selfstudy* which are partially related. CBOW model produced more relatable words than Skipgram (*study, coordinator* as opposed to *level, is, not*)

5. Assessment-related relationships:
   In *"exam : test :: assignment : ?"*, both models produced words like *interview* and *written*, which are related to evaluation methods. Although not exact matches, they still reflect the broader concept of assessment.

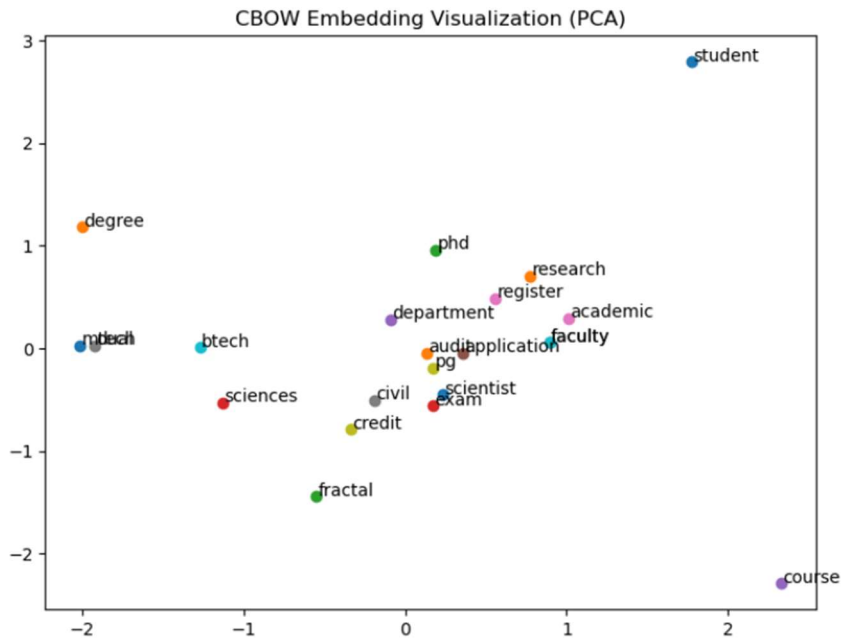
6. Program-specific associations:
   For *"phd : fellowship :: mtech : ?"*, the outputs included domain-related terms such as *engineering* and *materials*. This suggests that the model associates mtech and fellowship with their respective academic fields rather than exact parallel structures.

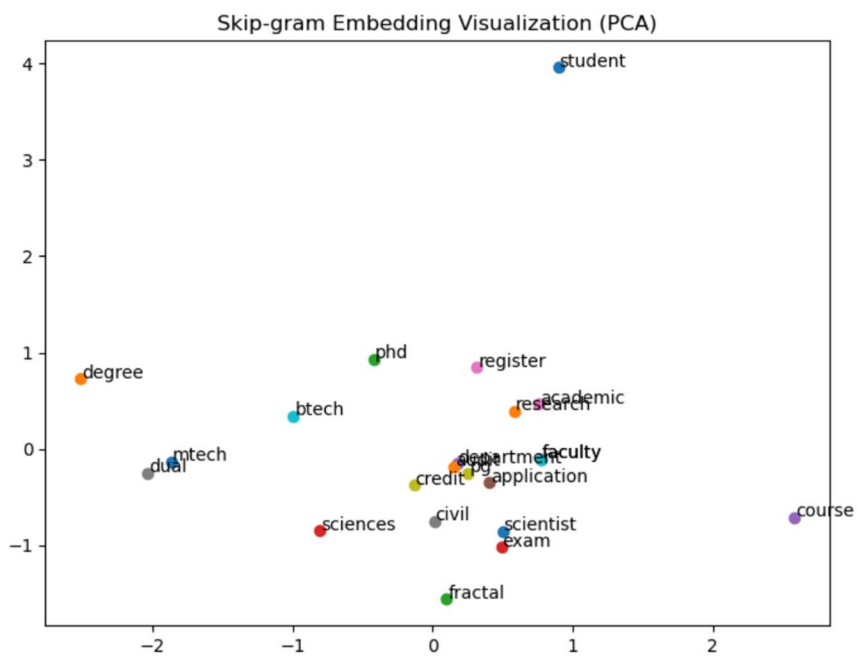
7. Admission Exam pathways:
   In *"jee : btech :: gate : ?"* and conversely *"btech : jee :: mtech : ?"* both models returned terms such as *programme* and *specialization.* Only the term *compulsory* was partially related which was generated in the second analogy

8. Contextual limitations:
   Some outputs (e.g., *cr, not, shall, is*) reflect frequent words in the dataset rather than meaningful analogies. This is likely because the corpus is derived from a university website, where such terms appear repeatedly in formal or instructional text.

6 Visualizations

Principal Component Analysis (PCA) was used to project selected word embeddings into a 2D space for visualization of clustering behaviour.





CBOW shows more broader and smoother clusters. "mtech" and "dual" appear very close, which is true, given the word "dual" always appears after "mtech" in the corpus.

Skip-gram shows more tighter clusters. The words "mtech" and "dual" appear together but with a little space in Skip-gram.

"Course" and "student" appear isolated in both CBOW and Skip-gram.

The words "credit" and "audit" appear at more distance from each other in CBOW as compared to Skip-gram.

Report for problem 2

Character-Level Name Generation using RNN Variants

1 Dataset Preparation

1000 unique indian names were generated using LLM (Perplexity). The names are both male and female in equal proportions and from all parts of the country. These names were stored in a txt file.

1.1 Preprocessing steps

1. 56 Random filler names like "zizi", "fifi" ,"xave" were detected and were deleted manually.
2. New 150 new names were generated and stored in the data file.
3. Duplicates were removed.
4. All the text was then converted to lowercase, extra space was removed.
5. Start <START> and end <END> tokens were added to indicate sequence boundaries.
6. Names were split into individual characters.
7. A vocabulary of unique characters was created.
   Vocabulary: '<END>', '<START>', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'
   Vocabulary size: 28
8. Each character was mapped to a numerical index for model input.
   Example: anaya -> <START> a n a n y a <END> was mapped to:
   1, 2, 15, 2, 15, 27, 2, 0

2 Model Implementations

Three models were implemented from scratch:

2.1 Vanilla RNN

A simple recurrent neural network was used to model character sequences.

Architecture:
Embedding layer
RNN layer
Fully connected output layer

Hyperparameters used:

Embedding size: 64
Hidden size: 128

Number of layers: 1
Epochs: 25
Learning rate: 0.001

Total trainable parameters: 30236
Model size (MB): 0.1153411865234375

2.2 Bidirectional LSTM (BLSTM)

A bidirectional LSTM was used to capture both forward and backward dependencies in sequences.

BLSTM Experimentation and Observations

Initial Configuration

The BLSTM model was first trained with:
Embedding size: 64
Hidden size: 128

The training loss decreased rapidly:

Epoch 1: 7.24
Epoch 2: 2.27
Epoch 3: 0.36
...
Epoch 8: 0.016

Observation:

Although the loss decreased significantly, the generated outputs were not meaningful and showed signs of repetition. This indicates that the model was overfitting and becoming overconfident in its predictions.

Reduced Model Capacity

To address overfitting, the model size was reduced:
Embedding size: 32
Hidden size: 64

However, the model began to produce repetitive outputs such as:

vnv
zzzwwa
fffffffffn

Observation:

The model collapsed to high-frequency characters. This is supposedly a common issue in sequence generation where the model repeatedly predicts a single high-probability token to minimize loss, this shows the model is overconfident.

Regularization Attempt

To improve generalization:
Dropout (0.3) was added
Number of layers increased to 2

Training loss:

Epoch 1: 1436
Epoch 10: 694

Generated outputs:

tmib
eyt
odw
tah
som

Observation:

While repetition was reduced, the generated names became shorter and less realistic. This indicates that the model was regularized but struggled to capture meaningful name structures.

Final Architecture:

Embedding layer
2-layer Bidirectional LSTM
Dropout (0.3)
Fully connected layer

Hyperparameters:
Embedding size: 32
Hidden size: 64
Epochs: 10
Learning rate: 0.001
Dropout: 0.3
Number of layers: 2

Total trainable parameters: 602908
Model size (MB): 2.2999114990234375

2.3 RNN with Attention

An attention mechanism was added to a basic RNN to allow the model to focus on relevant parts of the sequence.

Architecture:
Embedding layer
RNN layer
Attention layer
Fully connected layer

Hyperparameters:
Embedding size: 64
Hidden size: 128
Epochs: 10
Number of layers: 1
Learning rate: 0.001

Total trainable parameters: 50332
Model size (MB): 0.1920013427734375

3 Quantitative Analysis

Final training losses for the three models

Model	Loss
Vanilla RNN	1471.58
BLSTM	694.46
RNN + Attention	967.199

Two metrics were used for quantitative analysis

3.1 Novelty Rate

Percentage of generated names not present in the training set.

3.2 Diversity

Ratio of unique generated names to total generated names.

Results

Model	Novelty Rate	Diversity
Vanilla RNN	0.98	0.93
BLSTM	1.00	0.98
RNN + Attention	1.00	0.99

The higher novelty rates of BLSTM compared to the Vanilla RNN (0.98), indicate that it generates entirely new names rather than memorizing the training data. The RNN + Attention mechanism also showed the highest diversity (0.99), suggesting a wide range of outputs. However, the Vanilla RNN maintained competitive diversity (0.93) while producing more structured outputs compared to BLSTM and RNN with attention mechanism. This indicates that quantitative metrics alone are insufficient to evaluate generative quality.

4 Qualitative Analysis

4.1 Vanilla RNN

Sample outputs:

shamu
shikina
gudeshidesha
adeshabhadeshinumomo

Observations:

1. Generates relatively realistic and pronounceable names
2. Captures common phonetic patterns such as "sha", "na", "ka"
3. Produces longer sequences with some repetition
4. Struggles with proper stopping and sometimes generates overly long names

4.2 BLSTM

Sample outputs:

vhv
kul
vqy
rmb

Observations:

1. Produces very short and fragmented outputs
2. Poor quality results than RNN
3. High novelty and diversity but poor realism
4. Indicates overfitting and lack of meaningful sequence generation
5. Model converges quickly but fails to generate structured names

4.3 RNN with Attention

Sample outputs:

Gotisuubisupp
ghhubbccbopp
ppr
bbhhsushh
rraallmnn

Observations:

1. Shows slightly better structure than BLSTM
2. Generates longer sequences than BLSTM
3. Still suffers from repetition and unnatural character combinations
4. Attention helps but does not fully solve sequence modeling issues

5 Realism of Generated names

The realism of generated names was evaluated by observing how closely the outputs resembled actual Indian names in terms of phonetic structure, length, and pronounceability.

Despite lower quantitative scores, the Vanilla RNN generated more realistic and pronounceable names, capturing common phonetic patterns such as “sha”, “na”, and “ka” of Indian names. Some examples include:

shamu
shikina
gudeshidesha

These names, although not always perfect, were generally pronounceable and resembled real names. However, the model sometimes generated overly long names or repeated certain syllables, indicating limited control over sequence length and repetition.

In contrast, the BLSTM model, although highly diverse, produced short and fragmented outputs lacking meaningful structure, such as such as:

vhv
vqy
rmb

While the model showed high novelty and diversity, the outputs lacked meaningful structure and were not easily pronounceable. This suggests that the model either overfit the training data or failed to generalize meaningful patterns under the given configuration.

The Attention-based model showed some improvement in sequence modeling but still suffered from repetitive characters and unrealistic outputs. It generated longer sequences and occasionally reflected partial patterns seen in real names:

ceerinnatyy
ppttinn

However, the outputs still contained repeated characters and unnatural combinations, reducing overall realism.

6 Failure Modes

6.1 Vanilla RNN

1. Repetition of syllables or patterns
   The model often repeats certain character sequences:
   gudeshidesha
   shaleshalesha
2. Excessively long names
   Some generated names are longer than typical Indian names
   gutnshiriqaliqaliqa

6.2 BLSTM

1. Very short or incomplete names
   The model frequently generates names that are too short:
   vhv
   vqy
   These are not meaningful or realistic.
2. Random character combinations
   Outputs often lack phonetic structure:
   rmb
   gpq
   Names are unpronounceable and nothing like Indian phonetics.

6.3 RNN + Attention

1. Character repetition
   The model produces repeated characters:
   pppp
   eeee
   No readability and realism.
2. Unnatural character combinations
   Generated names contain unlikely sequences:
   zhhyjjohhwwx
   These do not resemble real names.
3. Inconsistent structure
   Some names are too short while others are overly repetitive:
   ee
   ppttinn

7 Challenges Faced

1. Repetition of characters in generated outputs
2. Difficulty in controlling sequence termination
3. Overfitting in BLSTM despite low loss
4. Handling special tokens correctly during generation